

Week 5 Day 2 Write-up: Raw Python vs. LangChain Agent

Raw Python Agent vs. LangChain Agent

On Day 1, I built an agent completely from scratch using a Python while loop and the OpenAI-compatible API. I had to manually define the tool schemas, detect whether the model requested a tool, execute the correct Python function, append the tool result back into the conversation, and repeat this process until the model returned a final answer. Although this required more code, it helped me understand exactly how an agent works internally through the ReAct (**Reason → Act → Observe**) loop.

On Day 2, I rebuilt the same idea using LangChain. Instead of managing the loop myself, LangChain handled most of the orchestration through its agent framework. I only needed to define my tools using the `@tool` decorator, provide them to the agent, and let the AgentExecutor manage tool selection, execution, and message history automatically. This reduced the amount of boilerplate code and made the implementation much cleaner.

The biggest difference is the level of abstraction. With raw Python, every reasoning step and API interaction was visible because I implemented each one myself. With LangChain, much of that complexity is hidden behind the framework. This makes development faster, but it also means debugging can be more difficult because the framework performs many operations internally.

Annotated Reasoning Trace

Example task: *"Compare the weather in Lahore and Karachi, then tell me which city is warmer."*

Step	Agent Action	ReAct Stage
1	Receives the user's request and determines that weather information is required.	Reason
2	Calls the <code>weather_lookup</code> tool for Lahore .	Act
3	Receives the result: 35°C, Sunny .	Observe
4	Calls the <code>weather_lookup</code> tool for Karachi .	Act
5	Receives the result: 32°C, Cloudy .	Observe
6	Compares both temperatures and concludes that Lahore is warmer before generating the final response.	Reason → Final Answer

Comparison of Reasoning Traces

The reasoning flow in both implementations follows the same ReAct pattern:

Reason → Act → Observe → Repeat → Final Answer

The difference is how much of the process is visible.

- In the **raw Python agent**, every step was explicitly implemented, including tool detection, tool execution, conversation updates, and iteration control. My logging statements clearly showed each reasoning step, tool call, and observation.
- In the **LangChain agent**, the reasoning flow is still the same, but the framework performs most of these operations internally. Using `verbose=True` exposes the execution trace, but the underlying message management and loop control are abstracted away.

Overall, LangChain made the agent significantly easier to build by handling the repetitive orchestration code, while the raw Python implementation provided a much deeper understanding of how tool-calling agents actually work behind the scenes.